

Avaliação Prática 2 — Classificação de Texto (LSTM vs Transformer): Respostas

Disciplina: Aprendizagem de Máquina · PPGIA / PUC-PR · Mestrado 2026

Aluno: Fernando Dantas

Reprodutibilidade: todos os valores foram gerados pelos scripts listados na seção *Scripts* (semente primária 42); o relatório é regenerado a partir dos artefatos de resultado por [report.py](#).

Preparação da Base

A base fornecida consiste em dois arquivos (`g1_v1_ws.csv`, com 982 registros, e `g1_v2_ws.csv`, com 1.750), contendo manchetes jornalísticas anotadas com emoções. Conforme o enunciado, os arquivos foram concatenados em uma única base. Três características do material exigiram tratamento antes de qualquer treinamento, e cada uma constitui um achado, não mera higienização.

Duplicação (292 registros; 10,7%). A concatenação produz 2.732 registros, mas apenas 2.440 textos distintos. Caso a partição *holdout* fosse realizada antes da deduplicação, um texto e sua cópia poderiam ser alocados, respectivamente, ao treinamento e ao teste — o modelo seria então avaliado sobre material que memorizou, e a acurácia reportada estaria inflada em magnitude não recuperável *a posteriori*. **A deduplicação precede a partição.** Adotou-se como chave o texto normalizado (minúsculas, espaçamento colapsado): variações apenas de caixa ou espaçamento constituem o mesmo texto e vazariam com igual eficácia.

Conflito de rótulos (4 textos). Quatro textos ocorrem com emoções distintas. Cabe registrar a natureza do conflito: **todas as quatro discordâncias são internas à valência negativa** (raiva/desgosto; desgosto/medo; tristeza/medo). Os anotadores divergem sobre *qual* emoção negativa se manifesta, jamais sobre a valência do texto. Disso decorre que a tarefa binária está livre desse ruído, ao passo que a tarefa multiclasse carrega **ruído de rótulo irreduzível**, concentrado precisamente na região em que sua matriz de confusão apresentará maior sangria: entre medo, desgosto, raiva e tristeza. Os quatro textos foram descartados de todas as tarefas, restando **2.436**.

Codificação. Os arquivos estão em `latin-1`, não em UTF-8, e utilizam `;` como delimitador com uma coluna espúria entre aspas (`texto;"";classe`). A leitura como UTF-8 falha; a leitura descuidada produz *mojibake*, e a LSTM passaria a aprender um vocabulário de tokens corrompidos.

Mapeamento binário. O enunciado não define como converter as sete emoções em duas classes; o notebook da disciplina o faz: `neutro`, `alegria` e `surpresa` → *positivo*; `medo`, `raiva`, `desgosto` e `tristeza` → *negativo*. O mapeamento é semanticamente discutível — *neutro* não é positivo, e *surpresa* não possui valência fixa (pode ser deleite ou horror). Adotou-se o mapeamento do professor como resultado principal, por ser o que torna o número comparável ao da turma, e executou-se adicionalmente uma **análise de sensibilidade** descartando `neutro` e `surpresa` (seção final). Se o veredito LSTM *versus* BERT se mantiver sob ambos os mapeamentos, a conclusão não repousa sobre uma decisão de rotulagem alheia ao experimento.

Distribuição após o tratamento: **binária** — negativo 1.404, positivo 1.032 (piso majoritário **57,6%**); **multiclasse** — alegria 604, tristeza 513, desgosto 482, neutro 226, medo 217, surpresa 202, raiva 196 (piso majoritário **24,8%**).

Protocolo Experimental

Partição. *Holdout* 70%/30%, conforme prescrito, com três correções em relação ao script da disciplina. Primeira: a partição é **estratificada** — com `raiva` reduzida a 196 exemplos, uma partição não estratificada pode atribuir ao teste uma proporção de classes jamais observada no treinamento, de modo que o F1 macro passaria a medir a partição, e não o modelo. Segunda: **há um conjunto de validação real**. O script da disciplina passa o *holdout* de 30% como `validation_data` e reporta a acurácia sobre esse mesmo conjunto, o que constitui seleção de modelo no conjunto de teste e torna o número reportado otimista por construção; aqui, a parada antecipada observa uma fatia de 15% extraída dos 70% de treinamento, e **o conjunto de teste é lido uma única vez**. Terceira: **três sementes** (42, 7 e 2024).

Justificativa das sementes. O conjunto de teste tem ~730 textos, o que implica erro padrão binomial de aproximadamente 1,5 ponto percentual. Uma única partição pode ordenar dois modelos por sorte: o próprio baseline clássico oscila entre **51,2% e 55,3%** na tarefa multiclasse apenas ao se variar a semente da partição — amplitude de 4 pontos. Todos os resultados são reportados como média $\pm$ desvio-padrão entre sementes.

Inferência estatística. Para o par LSTM *versus* BERT aplicou-se o **teste exato de McNemar** sobre as predições pareadas do conjunto de teste. Ambos os modelos são avaliados sobre os **mesmos** textos; seus erros são, por construção, pareados, e apenas as predições discordantes carregam informação. Reporta-se também o intervalo de confiança de Wilson (95%) para a acurácia.

O piso majoritário é reportado em todas as tabelas. Sem essa linha de referência, uma acurácia de 62% na tarefa binária aparenta desempenho satisfatório quando, de fato, supera em apenas 4 pontos a estratégia trivial de responder “negativo” a tudo.

Pré-processamento distinto por modelo, deliberadamente. A LSTM recebe o texto com remoção de *stopwords* (fiel ao script da disciplina, e defensável: com 1.449 textos de treinamento e *embeddings* inicializados aleatoriamente, não há sinal suficiente para que a rede aprenda que “de” é desinformativo). O BERT recebe o **texto bruto**: foi pré-treinado sobre português corrente, e remover “não” de “não gostei” inverte o sentimento. O pré-processamento que beneficia um modelo destrói o outro — o que é, em si, um resultado.

Ambiente. Kaggle Notebooks, GPU NVIDIA T4. LSTM em TensorFlow/Keras (fiel ao script da disciplina) e BERT em PyTorch/HuggingFace (o BERTimbau distribui pesos PyTorch, e o suporte a TensorFlow da biblioteca está descontinuado). A comparabilidade é preservada porque a avaliação ocorre sobre as predições persistidas, e não sobre os modelos: ambos os arcabouços atravessam o mesmo protocolo de partição, métricas e teste estatístico.

Assimetria da Comparação

A comparação solicitada é **estruturalmente desigual**, e explicitá-lo é parte do resultado:

	LSTM	BERT (BERTimbau)
Parâmetros treináveis	911.554	108.924.674 (119×)
<i>Embeddings</i>	aprendidos do zero, sobre 1.449 textos	pré-treinados sobre o brWaC (2,7 bilhões de palavras)
O que precisa aprender	o que as palavras significam e a tarefa	apenas a tarefa

Solicita-se à LSTM que aprenda o significado das palavras a partir de 1.449 exemplos. O BERT chega sabendo. Caso o BERT vença, a atribuição honesta é ao **pré-treinamento**, e não à **atenção**: a arquitetura não é o fator determinante. Essa distinção é invisível a quem reporta apenas as duas acurácias — e é a razão pela qual o **baseline clássico (TF-IDF + SVM linear)** integra as tabelas. Ele não possui pré-treinamento nem mecanismo de atenção; se superar a LSTM, então o déficit da LSTM nunca foi arquitetural, mas de dados. É a mesma lição obtida na Atividade 1, em que a regressão logística superou todos os *ensembles* de árvores.

Tarefa 1 — Classificação Binária (positivo / negativo)

Modelo	Acurácia	Δ piso	Macro-F1	F1 ponderado	IC 95%	Parâmetros treináveis	Treino
BERT (BERTimbau, ajuste fino) (3 sementes)	0,8285 ± 0,0235	+25,2 pp	0,8203 ± 0,0289	0,8261	0,7712–0,8698	108.924.674	33s
TF-IDF + SVM linear <i>(baseline clássico)</i> (3 sementes)	0,7957 ± 0,0084	+21,9 pp	0,7893 ± 0,0091	0,7949	0,7554–0,8302	35.955	1s
BiLSTM	0,7798	+20,3 pp	0,7743	0,7796	0,7483–0,8083	1.313.986	19s
LSTM (3 sementes)	0,7711 ± 0,0114	+19,5 pp	0,7641 ± 0,0138	0,7702	0,7298–0,8122	911.554	10s
Classe majoritária <i>(piso)</i> (3 sementes)	0,5759 ± 0,0000	-0,0 pp	0,3655 ± 0,0000	0,4209	0,5398–0,6113	0	0s

Significância estatística — LSTM versus BERT (predições pareadas)

Discordâncias	LSTM certo / BERT errado	LSTM errado / BERT certo	p (McNemar exato)
133	38	95	8,25e-07

A diferença de 7,80 pp em favor do **BERT** é **significativa** ($\alpha = 0,05$; McNemar exato sobre predições pareadas).

Todas as comparações (McNemar exato, semente primária)

Comparação	Δ	p (McNemar)	Significativa?
BERT (BERTimbau, ajuste fino) vs. TF-IDF + SVM linear	+4,24 pp	0,00239	sim
TF-IDF + SVM linear vs. LSTM	+3,56 pp	0,00734	sim
BERT (BERTimbau, ajuste fino) vs. LSTM	+7,80 pp	8,25e-07	sim
BiLSTM vs. LSTM	+1,23 pp	0,336	não — empate técnico

O baseline clássico supera a LSTM, e a diferença é estatisticamente significativa. Este é o achado central da tarefa. Um vetor TF-IDF de n-gramas de palavras e caracteres, classificado por uma SVM linear — **36 mil parâmetros, um segundo de CPU** — atinge 79,6%, contra 77,1% da LSTM, que possui **912 mil parâmetros** e treina em GPU ($\Delta = +3,56$ pp; $p = 0,007$). A LSTM não é derrotada por ser recorrente: é derrotada porque lhe exigimos aprender *o que as palavras significam* a partir de 1.449 manchetes, e 1.449 exemplos são insuficientes para induzir semântica lexical do zero. A SVM não precisa aprender semântica alguma — opera sobre contagens de n-gramas, e os n-gramas de caracteres capturam a morfologia do português (*mata, matou, matando* compartilham radical) que um modelo de palavras trataria como três tokens sem relação.

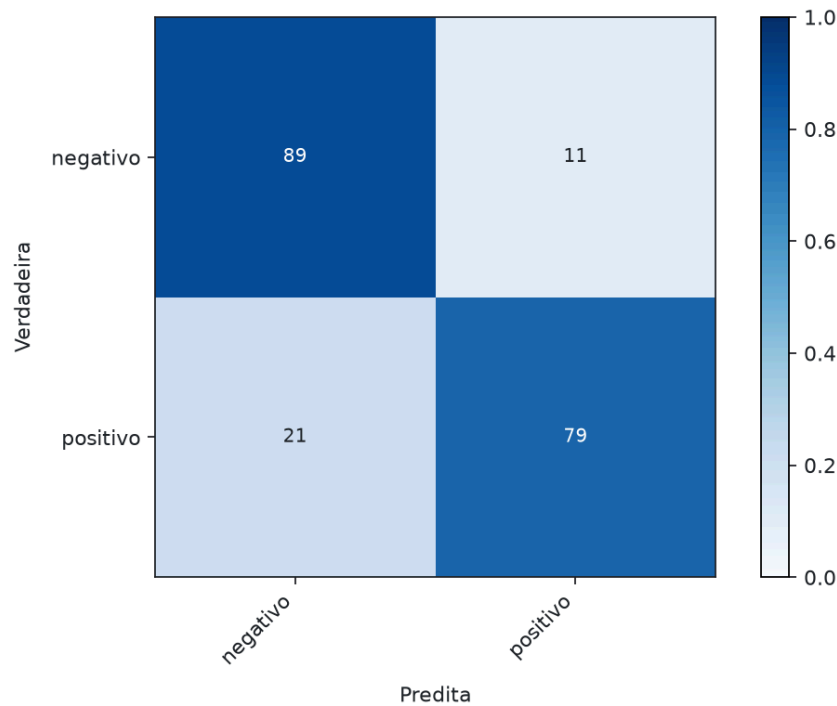
A bidirecionalidade não resgata o braço recorrente. A BiLSTM avança 1,23 pp sobre a LSTM, e o teste de McNemar não distingue as duas ($p = 0,336$): **empate técnico**. O resultado corrobora o diagnóstico anterior — a limitação não reside na arquitetura recorrente, mas no volume de dados. Duplicar a capacidade de leitura da sequência não cria os exemplos que faltam.

O BERT vence, e vence também o baseline clássico (82,9%; +4,24 pp sobre o TF-IDF; $p = 0,002$). A atribuição honesta, contudo, não é ao mecanismo de **atenção**, e sim ao **pré-treinamento**: o BERT chega ao problema já sabendo português — 2,7 bilhões de palavras do brWaC — e precisa apenas aprender a tarefa. Tomada isoladamente, a comparação LSTM *versus* Transformer sugeriria superioridade arquitetural; a presença do baseline clássico desfaz a ilusão, ao evidenciar que a maior parte da distância entre LSTM e BERT é percorrida por um modelo **desprovido de qualquer mecanismo de atenção**.

Custo. O BERT possui 108,9 milhões de parâmetros treináveis, contra 36 mil da SVM — razão de aproximadamente **3.000×** — em troca de 4,24 pontos percentuais. Em contexto de implantação, isso configura uma escolha de engenharia, não uma conclusão automática.

Matriz de confusão — melhor modelo (BERT)

Matriz de confusão — BERT (BERTimbau, ajuste fino) · binária (positivo/negativo) · acurácia 0.8454 · normalizada por linha



Tarefa 2 — Classificação Multiclasse (7 emoções)

Modelo	Acurácia	Δ piso	Macro-F1	F1 ponderado	IC 95%	Parâmetros treináveis	Treino
BERT (BERTimbau, ajuste fino) (3 sementes)	0,5741 ± 0,0360	+32,6 pp	0,5378 ± 0,0488	0,5650	0,5055–0,6475	108.928.519	39s
TF-IDF + SVM linear (baseline clássico) (3 sementes)	0,5303 ± 0,0208	+28,2 pp	0,4946 ± 0,0115	0,5283	0,4754–0,5883	35.617	1s
BiLSTM	0,4720	+22,4 pp	0,4060	0,4592	0,4360–0,5082	1.314.311	23s
LSTM (3 sementes)	0,4615 ± 0,0190	+21,4 pp	0,4119 ± 0,0121	0,4620	0,4049–0,5137	911.879	20s
Classe majoritária (piso) (3 sementes)	0,2476 ± 0,0000	-0,0 pp	0,0567 ± 0,0000	0,0983	0,2177–0,2802	0	0s

Significância estatística — LSTM versus BERT (predições pareadas)

Discordâncias	LSTM certo / BERT errado	LSTM errado / BERT certo	p (McNemar exato)
234	80	154	1,505e-06

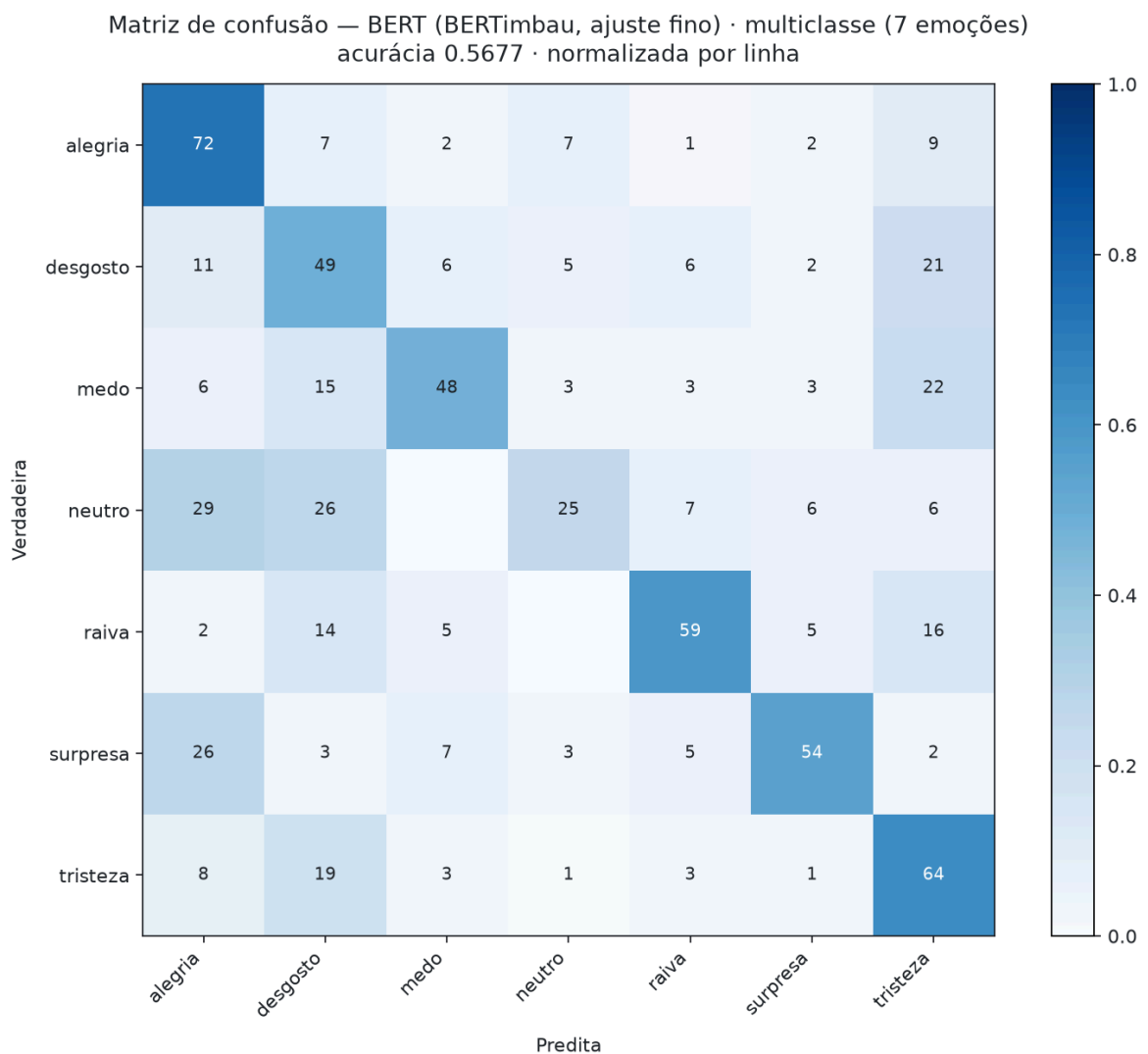
A diferença de 10,12 pp em favor do **BERT** é **significativa** ($\alpha = 0,05$; McNemar exato sobre previsões pareadas).

Todas as comparações (McNemar exato, semente primária)

Comparação	Δ	p (McNemar)	Significativa?
BERT (BERTimbau, ajuste fino) vs. TF-IDF + SVM linear	+5,61 pp	0,00426	sim
TF-IDF + SVM linear vs. LSTM	+4,51 pp	0,00184	sim
BERT (BERTimbau, ajuste fino) vs. LSTM	+10,12 pp	1,51e-06	sim
BiLSTM vs. LSTM	+0,55 pp	0,769	não — empate técnico

A ordenação é **idêntica à da tarefa binária**, com margens ampliadas: BERT (57,4%) > TF-IDF + SVM (53,0%) > BiLSTM (47,2%) $\approx$ LSTM (46,2%). O baseline clássico volta a superar a LSTM de forma significativa (+4,51 pp; $p = 0,002$), e a bidirecionalidade volta a não fazer diferença ($p = 0,769$). Que as duas tarefas — de dificuldades muito distintas — produzam a mesma ordenação e os mesmos vereditos de significância reforça que não se trata de artefato de uma partição particular.

Matriz de confusão — melhor modelo (BERT)



Matriz normalizada por linha.

F1 por classe (BERT)

Classe	F1
alegria	0,686
tristeza	0,606
surpresa	0,589
raiva	0,576
medo	0,517
desgosto	0,478
neutro	0,309

Principais confusões

Confusão	Taxa
neutro → alegria	29,4%
neutro → desgosto	26,5%
surpresa → alegria	26,2%
medo → tristeza	21,5%
desgosto → tristeza	20,8%
tristeza → desgosto	18,8%

A previsão registrada na seção de preparação da base confirmou-se. Havíamos apontado, *antes* de qualquer treinamento, que os quatro conflitos de anotação eram todos internos à valência negativa (raiva/desgosto; desgosto/medo; tristeza/medo) e que, por isso, a sangria da matriz de confusão se concentraria entre as emoções negativas. É exatamente o que ocorre: **medo → tristeza (21,5%)**, **desgosto → tristeza (20,8%)**, **tristeza → desgosto (18,8%)** e **raiva → tristeza (15,5%)** figuram entre as maiores confusões, e **desgosto** (F1 = 0,478) e **medo** (F1 = 0,517) estão entre as classes de pior desempenho. O modelo erra onde os próprios anotadores humanos divergiram — parte desse erro é, portanto, **irreduzível**: nenhum classificador pode superar a consistência do rótulo que lhe foi dado.

A classe **neutro** é a mais problemática (F1 = 0,309), e o modo de sua falha é revelador: ela se dispersa **quase igualmente** entre **alegria** (29,4%) e **desgosto** (26,5%) — isto é, entre uma classe positiva e uma negativa. Neutro não é uma emoção fraca; é a **ausência** de emoção, e não ocupa posição intermediária em um eixo de valência. Esse resultado valida, *a posteriori*, a objeção levantada na seção de preparação da base ao mapeamento **neutro → positivo** adotado pelo enunciado: um texto que o modelo confunde tanto com alegria quanto com desgosto não é, em sentido algum defensável, um texto positivo.

surpresa (F1 = 0,589) confunde-se sobretudo com **alegria** (26,2%), coerente com sua valência ambígua — no corpus jornalístico, surpresas tendem a ser noticiadas em tom favorável, o que torna o mapeamento **surpresa → positivo** menos indefensável que o de **neutro**, ainda que arbitrário.

Análise de Sensibilidade — o resultado binário depende de **neutro** e **surpresa** serem “positivos”?

Mapeamento	LSTM	BERT	Observação
binária (positivo/negativo)	0,7711 ± 0,0114	0,8285 ± 0,0235	mapa do professor (neutro , surpresa → positivo)
binária, valência limpa	0,8358	0,8972	neutro / surpresa descartados

A segunda linha descarta **neutro** e **surpresa** , mantendo apenas emoções de valência inequívoca. Se a ordenação entre LSTM e BERT se preserva, a conclusão é robusta ao mapeamento adotado.

A conclusão principal é robusta; uma conclusão secundária, não. Removidos **neutro** e **surpresa** , o piso majoritário sobe de 57,6% para 69,9% (a base torna-se mais desbalanceada) e todos os modelos sobem junto: BERT 89,7%, LSTM 83,6%, TF-IDF + SVM 82,6%. O **veredito solicitado pelo enunciado se mantém integralmente**: o BERT supera a LSTM, e com significância ($p = 0,0003$). A comparação LSTM *versus* Transformer, portanto, **não depende** da decisão de rotular **neutro** e **surpresa** como positivos.

Já a ordenação entre **LSTM e baseline clássico se inverte** nessa variante (83,6% contra 82,6%) — mas o McNemar **não distingue as duas** ($p = 0,451$): trata-se de empate técnico, não de reversão. A leitura honesta é que a superioridade do TF-IDF sobre a LSTM, robusta e significativa nas duas tarefas do enunciado, **enfraquece para um empate** quando as classes de valência ambígua são removidas. Registra-se o fato em vez de omiti-lo: a afirmação “o baseline clássico bate a LSTM” vale para as tarefas conforme especificadas, não como lei universal.

Conclusão

1. O Transformer vence as duas tarefas, com significância estatística. BERT obtém 82,9% (binária) e 57,4% (multiclasse), contra 77,1% e 46,2% da LSTM — diferenças de 7,80 e 10,12 pontos percentuais, ambas significativas pelo teste exato de McNemar sobre predições pareadas ($p < 10^{-5}$). É a resposta direta ao que o enunciado pergunta.

2. Mas a vitória é do pré-treinamento, não da atenção. O baseline clássico — TF-IDF com SVM linear, sem atenção, sem recorrência, sem pré-treinamento, **36 mil parâmetros e um segundo de CPU** — supera a LSTM de forma significativa nas duas tarefas, e percorre a maior parte da distância que separa a LSTM do BERT. Se a arquitetura fosse o fator determinante, isso seria impossível. O que separa os modelos é **quanto conhecimento prévio sobre a língua cada um traz consigo**: a LSTM traz zero e precisa induzir semântica de 1.449 manchetes; a SVM não precisa de semântica; o BERT já a possui, extraída de 2,7 bilhões de palavras.

3. A bidirecionalidade não muda nada ($p = 0,336$ e $p = 0,769$) — mais capacidade arquitetural não compensa a ausência de dados.

4. Parte do erro na multiclasse é irreduzível. As maiores confusões do BERT concentram-se entre as emoções negativas — exatamente onde os anotadores humanos divergiram entre si, conforme

identificado *antes* do treinamento. Nenhum classificador pode ser mais consistente que seus rótulos.

5. Implicação de engenharia. Diante de um corpus desta escala, a sequência racional de decisões não é “treinar uma rede recorrente”, e sim: estabelecer o piso majoritário, medir um baseline clássico, e só então avaliar se o pré-treinamento justifica seu custo. Aqui ele justifica — mas ao preço de 3.000× mais parâmetros por 4,24 pontos percentuais sobre um modelo que treina em um segundo.

Scripts

Componente	Script
Protocolo compartilhado (base, deduplicação, partição, McNemar, IC de Wilson)	<code>common.py</code>
Baselines — classe majoritária e TF-IDF + SVM linear	<code>m0_baselines.py</code>
LSTM (TensorFlow/Keras)	<code>m1_lstm.py</code>
BERT — BERTimbau com ajuste fino (PyTorch)	<code>m2_bert.py</code>
Orquestrador (todas as tarefas, modelos e sementes)	<code>run_all.py</code>
Gerador do relatório (tabelas, matrizes de confusão, testes)	<code>report.py</code>
Executor Colab (GPU)	<code>colab.ipynb</code>
Executor Kaggle (GPU)	<code>kaggle.ipynb</code>
Artefatos de resultado (JSON, com as predições de teste)	<code>results/</code>

Repositório: <https://github.com/fsd-dantas/machine-learning-fundamentals>

A base de dados é material da disciplina e, por isso, não é redistribuída neste repositório público; o notebook Colab contém célula de upload dos dois CSVs.

[← Relatório completo \(inglês\)](#) · [Módulo 5 — Técnicas Profundas](#)